

Fuzzing Libsixel

A report for CS-412

Maria Bennani
EPFL

Sean Perazzolo
EPFL

Valuthy Karunakaran
EPFL

Abstract

This report details the design, executions and analysis of a coverage guided fuzzing campaign on the open-source library `libsixel` using the `AFL++` fuzzer.

`libsixel` is an encoder/decoder library for `SIXEL`, an image format for printer and terminal imaging.

The project is comprised of two parts: white-box fuzzing (source-instrumented) and black-box (binary-only, using `QEMU`).

1 Harness Design

1.1 Selected Entrypoint

For this campaign, we selected the encoding path of the library, specifically targetting the `sixel_encoder_encode` function rather than the decoding path suggested by the lab guidelines. Indeed, we judged that the encoder represents a larger and more critical attack surface, as applications usually rely on `libsixel` to process binary image formats (e.g. `PNG`, `JPEG`, `GIF`) to output `SIXEL`. This makes the encoder a highly attractiv target. Additionally, recently identified vulnerabilities were due to bugs in the encoding path.

1.2 Considered Alternatives

We initially considered fuzzing the high-level `img2sixel` function. However, we rejected this approach because high-level wrappers introduce unnecessary overhead (e.g. command-line argument parsing, I/O operations, ...). By writing a custom harness directly around `sixel_encoder_encode`, we skip all non-encoding related functionality. **TODO: WHY WOULD IT BE (NOT) WORTH FUZZING THE DECODE PATH**

1.3 Data Flow and Guards

TODO: Then walk through your harness code and explain the data flow from the `AFL++` input file to the library API call. For every guard in the code (error handling, dimension limits, etc.), explain why it is needed from a fuzzing perspective.

2 Instrumentation and Sanitizers

List every compiler flag and patch you applied to the target library. For each one, explain what it does and what would happen if you omitted it. If you patched the library (e.g., removing checksums), explain the effect on path discovery.

3 Seed Corpus and Dictionary

Describe your seeds and dictionary. Using the dictionary and `havoc/splice` rows from your `AFL++` status screen, quantify how many new paths each strategy contributed. Explain what the dictionary entries represent in formal terms (hint: think grammar/language theory).

4 Campaign Analysis

Include your `afl-plot` edges graph and `AFL++` status screen screenshot. Report stability, corpus count, map density, and cycles done. Describe the shape of the edges curve and argue whether the campaign reached saturation.

5 Crash Triage

If crashes were found: Pick one crash and show the full triage – reproduce it, minimize it with `afl-tmin`, obtain an `ASan` stack trace. Identify the bug type and, if applicable, the corresponding CVE. If no crashes were found: Prove your

setup works by injecting a synthetic bug (e.g., an off-by-one write), re-fuzzing for 60 seconds, and showing AFL++ catches it. Then argue why no real bugs were found.

6 Attack Surface Analysis

Name two real-world applications that use your target library and describe a concrete attack scenario. Then identify at least two code paths in those applications that your harness does not exercise, and explain why these gaps matter for security.

7 Binary-Only Fuzzing with QEMU Mode

Run the same fuzzing campaign against an uninstrumented binary using AFL++ QEMU mode (-Q). Include the afl-plot edges graph and status screen for this campaign. Compare the two campaigns (instrumented vs. QEMU) on three axes: exec speed, edges discovered, and corpus count after the same wall-clock time. Explain why the numbers differ by describing how QEMU mode collects coverage compared to compile-time instrumentation.

We built uninstrumented versions of the harness and the library using gcc and g++ instead of afl-clang-lto and afl-clang-lto++ without passing the sanitizer options. AFL++ is then run with the -Q option and QASan enabled on this harness. We were able to reproduce the truevision bug due to the OOB read (TODO: insert link to the bug description).

8 Instrumentation Depth and Performance

Report the number of instrumented edges reported by afl-fuzz at startup for (a) the library alone and (b) the final harness binary. Explain why these numbers differ. Then compare your campaign's map density to the total instrumented edges and explain why not all edges were reached. Additionally, measure exec speed under three configurations using the same harness: (1) no sanitizer + fork mode, (2) ASan + fork mode, (3) ASan + persistent mode. Report the three numbers and explain the source of each speedup or slowdown.